

ui.notify 全面详解

ui.notify 是 NiceGUI 提供的全局通知组件，用于在界面角落显示临时消息提示（如操作结果、状态变更、警告信息等），支持自动关闭、自定义样式、图标、交互按钮等核心功能。其本质是 Quasar QNotify 组件的封装，核心特性包括轻量化调用、全局触发、多状态适配、灵活配置，是界面中反馈用户操作的核心组件。

一、核心概念与基础特性

1. 本质与用途

- 本质：全局浮动通知组件，无需提前声明 DOM 结构，通过函数调用即可在任意位置触发，默认显示在页面右上角（可配置位置）。
- 核心用途：为用户操作提供即时反馈，常见场景包括：
 - 操作结果：如“保存成功”“删除失败”“数据加载完成”；
 - 状态提示：如“已切换到深色模式”“网络连接恢复”；
 - 警告 / 错误：如“输入格式错误”“权限不足”“请求超时”；
 - 交互引导：如“请先完成登录”“文件上传中...”。
- 关键机制：
 - 显示逻辑：触发后自动弹出，默认 3 秒后自动关闭（可配置时长或禁用自动关闭）；
 - 堆叠行为：多个通知同时触发时自动堆叠排列，不会重叠；
 - 交互支持：可添加按钮实现二次操作（如“撤销删除”“查看详情”）；
 - 状态适配：内置成功、警告、错误、信息等预设状态，无需手动配置样式。

2. 基础用法

ui.notify 支持两种核心调用方式：**函数式调用**（推荐，简洁高效）和**类实例化**（支持更复杂配置），基础示例如下：

```
from nicegui import ui

# 1. 基础函数式调用（纯文本通知）
ui.button('基础通知', on_click=lambda: ui.notify('操作成功!'))

# 2. 带状态的通知（内置成功/警告/错误/信息状态）
ui.button('成功通知', on_click=lambda: ui.notify('数据保存成功', type='success'))
ui.button('警告通知', on_click=lambda: ui.notify('请检查输入格式', type='warning'))
ui.button('错误通知', on_click=lambda: ui.notify('网络请求失败', type='error'))
ui.button('信息通知', on_click=lambda: ui.notify('新消息提醒', type='info'))

# 3. 类实例化调用（支持更多配置）
ui.button('自定义通知', on_click=lambda: ui.Notification(
    message='带图标和延长时长的通知',
    icon='info',
    timeout=5000, # 5秒后关闭
    position='bottom-right' # 显示在右下角
).show())
```

```
ui.run()
```

二、核心配置参数（函数式调用）

函数式调用 `ui.notify(message, **kwargs)` 支持通过关键字参数配置所有特性，参数说明如下（基于官方文档核心配置，补充实战常用参数）：

参数名	类型	说明
message	str Element	通知内容（必填），支持纯文本或 NiceGUI 元素（如 <code>ui.html</code> 、 <code>ui.row</code> ）
type	str	通知状态类型（可选，默认 <code>'info'</code> ），支持值： <code>'success'</code> （绿色）、 <code>'warning'</code> （黄色）、 <code>'error'</code> （红色）、 <code>'info'</code> （蓝色）、 <code>'positive'</code> （同 success）、 <code>'negative'</code> （同 error）
icon	str	左侧图标（可选，支持 Quasar 图标库名称，如 <code>'check_circle'</code> 、 <code>'warning'</code> ，预设类型会自动匹配图标）
timeout	int None	自动关闭时长（单位：毫秒，默认 3000），设为 <code>None</code> 时永不自动关闭，需手动关闭
position	str	显示位置（可选，默认 <code>'top-right'</code> ），支持值： <code>'top-left'</code> / <code>'top-right'</code> / <code>'bottom-left'</code> / <code>'bottom-right'</code> / <code>'top'</code> / <code>'bottom'</code> / <code>'left'</code> / <code>'right'</code> / <code>'center'</code>
close_button	bool	是否显示关闭按钮（可选，默认 <code>False</code> ，自动关闭时无需显示； <code>timeout=None</code> 时建议设为 <code>True</code> ）
actions	List[Dict]	通知底部操作按钮（可选），格式： <code>[{'label': '按钮文本', 'on_click': 回调函数, 'color': '颜色'}]</code>
classes	str	CSS 类名（可选，支持 Tailwind、Quasar 类，如 <code>'bg-purple'</code> 、 <code>'text-white'</code> ）
props	str	Quasar 组件属性（可选，如 <code>'rounded-lg'</code> 圆角、 <code>'shadow-lg'</code> 阴影、 <code>'no-icon'</code> 隐藏图标）
style	str	内联 CSS 样式（可选，如 <code>'font-size: 14px; padding: 12px;'</code> ）

函数式调用示例（复杂配置）

```
from nicegui import ui

# 带操作按钮、自定义样式、无自动关闭的通知
def on_undo():
    ui.notify('已撤销删除操作', type='success')

ui.button('带按钮的通知', on_click=lambda: ui.notify(
    message='文件已删除，可在30秒内撤销',
    type='warning',
```

```

    icon='delete',
    timeout=None, # 不自动关闭
    close_button=True, # 显示关闭按钮
    position='bottom-center',
    actions=[
        {'label': '撤销', 'on_click': on_undo, 'color': 'blue'},
        {'label': '确认', 'on_click': lambda: ui.notify('已确认删除'), 'color': 'gray'}
    ],
    classes='bg-gray-50 text-gray-800 rounded-xl shadow-lg',
    style='padding: 16px;'
))

# 带复杂内容（HTML+图标）的通知
ui.button('复杂内容通知', on_click=lambda: ui.notify(
    message=ui.row(
        ui.icon('cloud_download', color='blue'),
        ui.html('<b>文件上传完成</b><br>已保存到「我的文档」文件夹')
    ),
    type='success',
    timeout=4000,
    position='top'
))

ui.run()

```

三、类实例化配置 (ui.Notification)

通过 `ui.Notification()` 类实例化可支持更精细的配置和方法调用，初始化参数与函数式调用一致，核心属性如下：

属性名	类型	说明
message	str Element	通知内容（可动态修改）
type	str	通知类型（可动态修改，如 <code>notification.type = 'success'</code> ）
icon	str	图标（可动态修改）
timeout	int None	自动关闭时长（可动态修改）
position	str	显示位置（可动态修改）
is_open	bool	通知显示状态（只读， <code>True</code> 为显示中， <code>False</code> 为已关闭）
classes	str	CSS 类名（可动态修改）
props	str	Quasar 组件属性（可动态修改）

类实例化示例（动态控制）

```
from nicegui import ui

# 动态修改通知内容和状态
def show_dynamic_notification():
    # 实例化通知（未显示）
    notification = ui.Notification(
        message='正在加载数据...',
        type='info',
        timeout=None,
        position='center'
    )
    notification.show() # 手动显示

    # 模拟异步操作（2秒后更新通知）
    ui.timer(2, lambda: (
        notification.set_message('数据加载完成! '),
        notification.type = 'success',
        notification.timeout = 2000 # 2秒后自动关闭
    ))

ui.button('动态通知', on_click=show_dynamic_notification)

# 手动关闭通知
def show_persistent_notification():
    notification = ui.Notification(
        message='需要手动关闭的通知',
        type='warning',
        timeout=None,
        close_button=True,
        position='bottom-left'
    ).show()

    # 外部按钮关闭
    ui.button('关闭通知', on_click=notification.close).classes('mt-2')

ui.button('持久化通知', on_click=show_persistent_notification)

ui.run()
```

四、核心方法（类实例化专用）

`ui.Notification` 实例提供以下方法用于动态控制通知：

方法名	作用	示例
show()	显示通知（实例化后需手动调用，函数式调用自动显示）	<code>notification.show()</code>
close()	手动关闭通知	<code>notification.close()</code>

方法名	作用	示例
set_message(content: str Element)	动态修改通知内容	<code>notification.set_message('新的通知内容')</code>
update()	强制更新通知状态（修改属性后调用，确保客户端同步）	<code>notification.update()</code>

方法调用示例

```
from nicegui import ui

notification = None

def create_notification():
    global notification
    notification = ui.Notification(
        message='初始内容',
        type='info',
        timeout=None,
        position='top-center'
    ).show()

def update_notification():
    if notification and notification.is_open:
        notification.set_message(ui.row(
            ui.icon('star', color='yellow'),
            ui.label('动态更新后的内容')
        ))
        notification.type = 'success'
        notification.classes('bg-green-50 text-green-800')
        notification.update()

ui.button('创建通知', on_click=create_notification)
ui.button('更新通知', on_click=update_notification).classes('m1-2')
ui.button('关闭通知', on_click=lambda: notification.close() if notification else None).classes('m1-2')

ui.run()
```

五、高级用法（实战场景）

1. 全局通知配置（统一样式）

通过 `ui.notify.config()` 配置全局默认参数，避免重复设置，适用于项目统一风格：

```
from nicegui import ui

# 全局通知配置（所有通知默认生效）
ui.notify.config(
    position='bottom-right',
```

```

        timeout=4000,
        classes='rounded-lg shadow-md',
        props='no-icon' # 全局隐藏图标
    )

# 后续调用无需重复配置
ui.button('全局样式通知1', on_click=lambda: ui.notify('全局配置生效', type='success'))
ui.button('全局样式通知2', on_click=lambda: ui.notify('无需重复设置位置和时长', type='info'))

# 局部配置可覆盖全局
ui.button('局部覆盖通知', on_click=lambda: ui.notify(
    '局部配置覆盖全局',
    position='top-center',
    timeout=6000,
    classes='bg-purple text-white'
))

ui.run()

```

2. 异步操作反馈（加载中→成功 / 失败）

结合异步函数（如 API 请求、数据处理），实现通知状态动态切换：

```

from nicegui import ui
import asyncio

async def async_task():
    # 显示加载中通知
    loading_notification = ui.Notification(
        message='正在处理数据...',
        icon='hourglass',
        type='info',
        timeout=None,
        position='center'
    ).show()

    # 模拟异步操作（3秒）
    await asyncio.sleep(3)

    # 关闭加载通知，显示结果通知
    loading_notification.close()
    ui.notify('数据处理完成', type='success', icon='check_circle')

ui.button('执行异步任务', on_click=async_task)

# 错误处理场景
async def async_task_with_error():
    loading_notification = ui.Notification(
        message='正在请求接口...',
        icon='cloud',
        type='info',
        timeout=None,

```

```

        position='center'
    ).show()

    try:
        await asyncio.sleep(2)
        raise Exception('网络超时') # 模拟错误
    except Exception as e:
        loading_notification.close()
        ui.notify(f'请求失败: {str(e)}', type='error', timeout=None, close_button=True)

ui.button('执行带错误的任务', on_click=async_task_with_error).classes('m1-2')

ui.run()

```

3. 自定义交互通知（带输入框 / 下拉框）

通知内容支持嵌套任意交互组件，实现复杂反馈逻辑（如“修改名称”“选择选项”）：

```

from nicegui import ui

def show_interactive_notification():
    def on_submit():
        if input_value.value:
            notification.close()
            ui.notify(f'已修改名称为: {input_value.value}', type='success')

    input_value = ui.input(placeholder='输入新名称')
    notification = ui.Notification(
        message=ui.column(
            ui.label('修改文件名称').classes('font-bold'),
            input_value,
            ui.button('确认', on_click=on_submit).classes('mt-2 bg-blue text-white')
        ),
        timeout=None,
        close_button=True,
        position='center',
        classes='w-64 bg-white p-4 rounded-xl shadow-lg'
    ).show()

ui.button('交互型通知', on_click=show_interactive_notification)

ui.run()

```

4. 品牌化样式定制（完全自定义）

结合 Tailwind CSS 和 Quasar props，实现与产品风格一致的通知样式：

```

from nicegui import ui

# 品牌化成功通知（渐变背景、自定义图标、圆角阴影）
ui.button('品牌化通知', on_click=lambda: ui.notify(
    message='恭喜！操作已完成',

```

```

    icon='celebration',
    type='success',
    position='top-center',
    timeout=3000,
    classes='bg-gradient-to-r from-blue-500 to-purple-600 text-white rounded-2xl shadow-xl p-4',
    style='font-size: 15px; font-weight: 500;',
    props='no-border'
  ))

# 极简风格通知（无图标、浅色背景）
ui.button('极简通知', on_click=lambda: ui.notify(
    message='操作提示',
    position='bottom-center',
    timeout=2000,
    classes='bg-gray-100 text-gray-800 rounded-lg p-2',
    props='no-icon no-shadow'
)).classes('m1-2')

ui.run()

```

六、注意事项

- 函数式 vs 类实例化：
 - 函数式调用 (`ui.notify(...)`)：简洁高效，适合简单通知，自动显示，无需手动管理生命周期；
 - 类实例化 (`ui.Notification(...)`)：支持动态修改、手动控制显示 / 关闭，适合复杂通知（如异步反馈、交互组件）。
- 自动关闭与关闭按钮：`timeout=None` 时建议设置 `close_button=True`，否则用户无法手动关闭通知；自动关闭时无需显示关闭按钮，避免冗余。
- 位置选择：根据使用场景选择位置（如操作结果通知用 `top-right`，重要提示用 `center`，底部操作反馈用 `bottom-center`），避免遮挡核心界面元素。
- 样式优先级：局部配置（函数 / 类参数）> 全局配置（`ui.notify.config()`）> 组件默认样式，可灵活覆盖。
- 性能注意：避免短时间内触发大量通知（如循环中调用），可能导致界面卡顿；可通过队列机制控制通知显示频率。
- 版本兼容性：`actions` 参数、`no-icon` 等 props 需 NiceGUI 1.2+ 版本支持，`ui.Notification` 类的 `set_message` 方法需 1.3+ 版本，使用时需确认版本匹配。
- 移动端适配：通知位置建议选择 `top / bottom / center`，避免 `left / right` 位置在小屏幕上被遮挡；可通过 `classes` 设置响应式宽度（如 `w-48 md:w-64`）。

通过以上配置与方法，`ui.notify` 可灵活满足从简单文本提示到复杂交互反馈的各类需求，是提升界面用户体验的核心组件之一。